

TP02 - Compte rendu - Processus, signaux et Journaux système

Année 2025 – 2026

Objectifs

Étudier les services UNIX, les processus, les signaux, les tubes (pipes) et le système de journalisation.

1. Secure Shell – SSH

1.1 Connexion SSH root (reprise TP01)

Connexion en root via la console du serveur puis installation du service SSH.

Commandes utilisées :
`apt search openssh`
`apt install openssh-server`

La configuration du serveur SSH se fait dans le fichier :
`/etc/ssh/sshd_config`

Le paramètre modifié est :
`PermitRootLogin yes`

Ce paramètre permet l'autorisation de la connexion root à distance par mot de passe.

Avantages :

- Accès administrateur direct
- Simplicité lors des phases de configuration

Inconvénients :

- Risque de sécurité en cas d'attaque brute-force

Redémarrage du service :
`systemctl restart ssh`

1.2 Authentification par clé – Génération de clés

Génération d'un couple de clés publique / privée sur la machine hôte.

Commande utilisée :
ssh-keygen

Dans le cadre du TP, aucune passphrase n'a été définie afin de simplifier la connexion.

Dans un contexte réel, ne pas utiliser de passphrase est une mauvaise pratique car toute personne ayant accès à la clé privée peut se connecter sans restriction.

Les clés publique et privée sont sauvegardées sur la machine hôte.

1.3 Authentification par clé – Connexion au serveur

Création du dossier .ssh dans /root s'il n'existe pas :
mkdir /root/.ssh

Création du fichier authorized_keys et ajout de la clé publique.

Droits appliqués :
Lecture / écriture / exécution pour le propriétaire uniquement, conformément aux exigences d'OpenSSH.

1.4 Connexion par clé depuis la machine hôte

Commande utilisée :
ssh -i maclef.pub root@ipserveur on utilise plutôt (dans mon cas ssh -i ~/.ssh/id_rsa root@IP_DU_SERVEUR)

La connexion s'effectue sans mot de passe grâce à la clé publique.

1.5 Sécurisation de l'accès SSH

Désactivation de l'authentification par mot de passe pour root afin d'éviter les attaques par brute-force.

Les attaques brute-force SSH consistent à tester automatiquement un grand nombre de mots de passe jusqu'à trouver le bon.

Autres méthodes de protection possibles :

- Utilisation de clés SSH uniquement
- Changement du port SSH

2. Processus UNIX

2.1 Étude des processus

Affichage des processus

Commande utilisée :

`ps auxww`

Colonnes demandées :

USER, PID, %CPU, %MEM, STAT, START, TIME, COMMAND

TIME correspond au temps CPU cumulé utilisé par le processus depuis son lancement.

Processus les plus consommateurs

Commande utilisée :

`top`

Dans mon cas, le processus le plus gourmand est le PID 1013 utilisant environ 33,3 % du CPU.

Premier processus lancé

Le premier processus est `init` ou `systemd` avec le PID 1.

Heure de démarrage du système

Commandes utilisées :

`uptime`

`who -b`

Nombre approximatif de processus depuis le démarrage

Commande utilisée :

`ps -e | wc -l`

`ps -e` affiche tous les processus système, `wc -l` permet de les compter.

2.2 Processus père (PPID)

Affichage du PID et du PPID :

`ps -eo pid,ppid`

Cette commande permet d'identifier pour chaque processus son processus père.

2.3 Arbre des processus

Commande utilisée :
pstree

pstree affiche l'ensemble des processus sous forme d'arbre hiérarchique.

2.4 Commande top et htop

Dans top :

- Touche M : tri par mémoire
- Touche P : tri par CPU
- Touche ? : aide

Le processus le plus gourmand en mémoire est observé via la colonne %MEM

Avantages de htop :

- Interface colorée
- Navigation facile
- Sélection des processus avec les flèches

Inconvénients :

- Consomme légèrement plus de ressources
- Pas toujours installé par défaut

3. Exercice 2 – Arrêt d'un processus

Scripts utilisés

Deux scripts ont été créés :

date.sh

date-toto.sh

Les scripts contiennent une boucle infinie affichant l'heure toutes les secondes.

Lancement

Commande :
./date.sh

Mise en arrière-plan :
CTRL-Z puis bg

Arrêt avec jobs

Commande :
jobs

Relance d'un script :
fg

Arrêt :
CTRL-C

Arrêt avec kill

Commande :
ps aux | grep date.sh
kill PID

sleep 1 permet de limiter la charge CPU.

4. Exercice 3 – Tubes (pipes)

Différence entre cat et tee

- cat affiche ou redirige un flux
- tee affiche le flux et l'écrit simultanément dans un fichier

Commandes étudiées

```
ls | cat  
ls -l | cat > liste  
ls -l | tee liste  
ls -l | tee liste | wc -l
```

5. Journal système – rsyslog

Vérification du service

Sur certaines distributions, rsyslog n'est pas installé par défaut.

Fichiers de logs

- /var/log/syslog
- /var/log/messages

Service cron

cron permet l'exécution automatique de commandes planifiées.

Surveillance des logs

Commande utilisée :

`tail -f /var/log/messages`

Lors du redémarrage de cron depuis un autre shell, des messages apparaissent en temps réel.

Logrotate

Le fichier `/etc/logrotate.conf` permet la rotation, la compression et la suppression automatique des logs.

dmesg

La commande `dmesg` affiche les messages du noyau, notamment le processeur détecté et les cartes réseau.

Problèmes rencontrés

- Service `rsyslog` absent
- Commandes dépendantes de la distribution

Conclusion

Ce TP a permis de comprendre le fonctionnement des services SSH, la gestion des processus, des signaux, des tubes et des journaux système sous UNIX.

Sources

- `man ps`, `top`, `kill`, `pstree`
- <https://linux.die.net>
- <https://tldp.org>